

CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

C Composite Data Structures

Arrays

- Static Arrays
- Array Syntax
- Arrays as Function Arguments

Arrays Are a Collection of a Single Type

An **array** is an ordered collection of data elements of the same type.

Each element is in a specific position and can be accessed via indexing (index starts at 0).



Static Arrays Have a Fixed Capacity

Statically declared arrays have a fixed capacity that is defined when the variable is declared.

```
<typename> <varname>[<capacity>;
```

```
int nums[10];
```

Static Arrays Have a Fixed Capacity

Statically declared arrays have a fixed capacity that is defined when the variable is declared.

```
<typename> <varname>[<capacity>;
```

```
int nums[10];
```

☆ We will cover dynamically allocated arrays later ☆

C Array Syntax

Declare empty array:

```
<typename> <varname>[<capacity>;  
    int nums[10];
```

Declare and initialize array:

```
<typename> <varname>[] = {<element list>;  
    int nums[] = {1, 2, 3};
```

Access array element:

```
<varname>[<element index>]  
    nums[1]
```

C Arrays Have No Boundary Checks

An array is stored in memory with other data around it.



It is legal to access positions outside the bounds of your array, but the behavior is unspecified.

`arr[6] = ???`

`arr[7] = ???`

`arr[-1] = ???`

C Arrays Don't Have Built-In Functions

C only supports creating and indexing an array, anything else must be implemented by programmer.

No length, sort, binarySearch, compare, etc.

Passing Arrays as Function Arguments

The array variable is equivalent to the *base address* of the array (the memory location of the first element).

When an array is passed to a function as an argument the function parameter is a *copy of the base address*, so modifying the array parameter modifies the original argument.

```
void main() {  
    int arr[] = {1, 2, 3};  
    int size = 3;  
    clear(arr, size);  
    // what is the  
    // value of arr?  
}
```

```
void clear(int a[], int size) {  
    for(int i=0; i<size; i++) {  
        a[i] = 0;  
    }  
}
```

Passing Arrays as Function Arguments

The array variable is equivalent to the *base address* of the array (the memory location of the first element).

When an array is passed to a function as an argument the function parameter is a *copy of the base address*, so modifying the array parameter modifies the original argument.

```
void main() {  
    int arr[] = {1, 2, 3};  
    int size = 3;  
    clear(arr, size);  
    // what is the  
    // value of arr?  
}
```

➡ {0, 0, 0}

```
void clear(int a[], int size) {  
    for(int i=0; i<size; i++) {  
        a[i] = 0;  
    }  
}
```

Demo: arrays.c

Strings

- Character Arrays
- String Library

C Does Not Have a String Type

Strings are implemented as arrays of char values terminated with the null character ('\0').

```
char string1[10];
```

C Does Not Have a String Type

Strings are implemented as arrays of char values terminated with the null character ('\0').

```
char string1[10];
```

☆ need to declare char array with enough capacity to store each character and null

String Length $\neq$ Array Length

The length of a string is the number of characters before the null character.

The capacity of the character array may be longer, but cannot be shorter, than the length of the string it contains.

C Has a String Library

The C string library contains many useful string functions.

To use the string library: `#include<string.h>`

String length: `int len = strlen(str)`

Copy string: `strcpy(copy, original)`

C Has a String Library

The C string library contains many useful string functions.

To use the string library: `#include<string.h>`

String length: `int len = strlen(str)`

Copy string: `strcpy(copy, original)`

- ☆ Programmer is responsible for passing well-formed strings as arguments
 - null-terminated
 - enough capacity

String Literal

A **string literal** is a fixed sequence of characters stored in global memory automatically ended with a null terminator.

“Mary”

String Literal

A **string literal** is a fixed sequence of characters stored in global memory automatically ended with a null terminator.

`"Mary"`

You can initialize a character array with a string literal:

```
char name[10] = "Mary";
```

String Literal

A **string literal** is a fixed sequence of characters stored in global memory automatically ended with a null terminator.

`"Mary"`

You can initialize a character array with a string literal:

```
char name[10] = "Mary";
```

You cannot assign a character array with a string literal:

```
char name[10];  
name = "Mary";
```

Exercise: vowels.c

Structs

- Struct Syntax
- Using Structs
- Structs as Function Arguments

Structs Are a Collection of Different Types

A **struct** is a collection of data elements of different types that is treated as a single unit.

```
struct Student {  
    int year;  
    char major[32];  
};
```

Defining and Declaring Structs

Structs are defined outside of any function or in a header file:

```
struct <struct_name> {  
    <field 1 type> <field 1 name>;  
    <field 2 type> <field 2 name>;  
    <field 3 type> <field 3 name>;  
    ...  
};
```

Declare a variable of the struct type with struct keyword:

```
struct <struct_name> <var_name>;
```


Declaring Initialized Structs

You can also declare and initialize a struct at the same time.

```
struct <struct_name> {  
    <field 1 type> <field 1 name>;  
    <field 2 type> <field 2 name>;  
    <field 3 type> <field 3 name>;  
    ...  
};
```

```
struct <struct_name> <var_name> = {<field list>;};
```

Fields Are Accessed Using Dot Notation

```
struct Student{  
    int grad_year;  
    char major[32];  
};
```

```
void main(){  
    struct Student st1;  
    st1.grad_year = 2028;  
    strcpy(st1.major, "chem");  
}
```

variable name

field name

Passing Structs as Function Arguments

When a struct is passed to a function as an argument the function parameter is a *copy of each of the fields*, so modifying the struct parameter does not modify the original argument.

```
struct Student{  
    int grad_year;  
    char major[32];  
};  
  
void updateMajor (  
    struct Student s) {  
    strcpy(s.major, "cs");  
}
```

```
void main(){  
    struct Student st1;  
    st1.grad_year = 2028;  
    strcpy(st1.major, "chem");  
  
    updateMajor(st1);  
    // what is the value  
    // of st1.major  
}
```

Passing Structs as Function Arguments

When a struct is passed to a function as an argument the function parameter is a *copy of each of the fields*, so modifying the struct parameter does not modify the original argument.

```
struct Student{  
    int grad_year;  
    char major[32];  
};  
  
void updateMajor (  
    struct Student s) {  
    strcpy(s.major, "cs");  
}
```

```
void main(){  
    struct Student st1;  
    st1.grad_year = 2028;  
    strcpy(st1.major, "chem");  
  
    updateMajor(st1);  
    // what is the value  
    // of st1.major  
}
```

 "chem"

Demo: point.c